

Natural Language Processing

Lecture 17: Semantic Parsing -
Formal Semantics and CCG

12/5 & 12/7/2022

COMS W4705
Daniel Bauer

Semantic Parsing

- Goal : Convert raw input text into some meaning representation.
- vs. Semantic Role Labeling (SRL):
SRL only considers individual predicate-argument structures.
- Today we will consider meaning representations for complete sentences.

Semantic Parsing Questions

- What should the target representations be?
- Can we compute sentence meaning from lexical meaning?
- What is the role of syntax?
- What algorithms and resources (training data etc.) do we need for semantic parsing?

Goals for Meaning Representations

- Should be **unambiguous** (resolve any ambiguity [syntactic ambiguity, word-sense etc.] present in the natural language utterance).
- **Canonical form:** Different sentences that have the same meaning should have the same canonical meaning representation.
- **Support inference:** Want to use meaning representation to draw logical conclusions.
- **Expressiveness:** Want to capture a wide range of semantic phenomena and subject matter.

First-Order Logic (FOL)

- Traditional approach to meaning representation ("Logical Form")
- Formal properties are well understood. Efficient algorithms for inference.
- First-order logic is written in one particular formal language, can describe this language using a CFG.
- Components of a FOL formula: **Constants, Functions, Terms, Quantifiers, Variables, Logical Connectives.**
- Model-theoretic semantics: A formula is "true" in a particular set of models (truth conditions).

Terms

- Constants refer to exactly one specific entities in the world.

ColumbiaUniversity, Alice

- Functions map a constant to a specific other constant.

Age(Alice), LocationOf(ColumbiaUniversity), Age(MotherOf(Alice))

- Variables are used to make generalizations over sets of objects in the world. (a term that does not contain a variable is called a "ground" term)

Predicates

- Predicates are used to make statements over terms

Student(Alice)

Alice is a student.

Attends(Alice, ColumbiaUniversity)

Alice goes to Columbia.

Love(Mother(Alice), Alice)

Alice's mother loves her.

- A predicate applied to a list of terms is an **atomic FOL-formula**.

Logical Connectives

- Logical connectives $\neg$, $\wedge$, $\vee$, $\rightarrow$, are used to build formulas from atomic formulas.

$Student(Alice) \wedge Attends(Alice, ColumbiaUniversity)$

Alice is a student who attends Columbia

$\neg Love(Mother(Alice), Alice)$

Alice's mother does not love her.

$Attends(Alice, ColumbiaUniversity) \rightarrow Student(Alice)$

Alice attends Columbia, therefore she is a student.

$Attends(Alice, ColumbiaUniversity) \vee Attends(Alice, CityCollege)$

Alice attends Columbia or CityCollege.

Variables and Quantifiers

- Terms may contain variables. These are used in two ways: Existential quantification and universal quantification.
- Existentially quantified formulas assert that some entity exists for which the formula is true.

$\exists x. \text{Attends}(x, \text{ColumbiaUniversity})$
There is someone who attends Columbia.

- Universally quantified formulas assert that the formula is true for all entities (in the world).

$\forall x. \text{Love}(\text{Mother}(x), x)$
Everyone is loved by their mother.

Variables and Quantifiers

- Existential quantification often appears with conjunction.

$\exists x \text{ Attends}(x, \text{ColumbiaUniversity}) \wedge \text{Likes}(x, \text{IceCream})$

Someone goes to Columbia who likes ice cream.

- Universal quantification often appears with implication.

$\forall x \text{ Attends}(x, \text{ColumbiaUniversity}) \rightarrow \text{Student}(x)$

Everyone who attends Columbia is a student.

CFG for FOL formulae

Formula $\rightarrow$ AtomicFormula
| Formula Connective Formula
| Quantifier Variable . Formula
| $\neg$ Formula
| (Formula)

AtomicFormula $\rightarrow$ Predicate(Term, ...)

Term $\rightarrow$ Function(Term)
| Constant
| Variable

Term $\rightarrow$ Function(Term)
| Constant
| Variable

Connective $\rightarrow \wedge \mid \vee \mid \rightarrow$

Quantifier $\rightarrow \exists \mid \forall$

Constant $\rightarrow$ *Alice* | *ColumbiaUniversity* |...

Variable $\rightarrow x \mid y \mid z \mid \dots$

Predicate $\rightarrow$ *Attends* | *Loves* | *Student* |...

Function $\rightarrow$ *Mother* | *Age* | ...

Model Theoretic Semantics

- The meaning of a formula is its truth conditions.
- Truth conditions can be described as a set of "possible worlds" (models) that make the expression true.
 - Such a model must contain all entities referred to by the terms of the formula.
 - For example

$$\forall x \textit{Attends}(x, \textit{ColumbiaUniversity}) \rightarrow \textit{Student}(x)$$

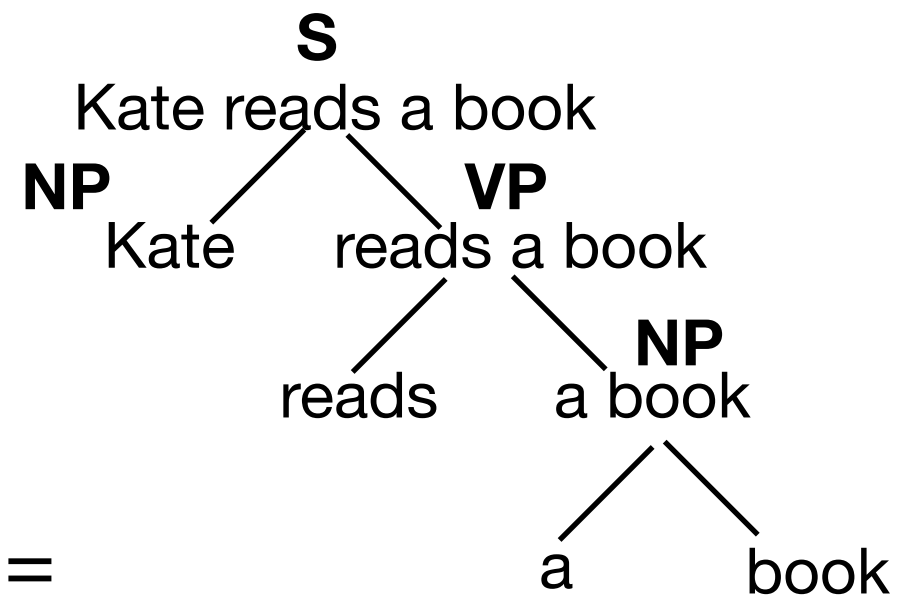
is true in all worlds in which everyone who attends Columbia is a student.

Principle of Compositionality

- The meaning of a complex expression should be completely determined by
 - the sub-parts of the expression
 - the rules used to combine these expressions.
- We should be able to compute sentence meaning from word meaning.

Compositionality

$[[\text{Kate reads a book}]] =$
 $C_1([[\text{Kate}]], [[\text{reads a book}]]) =$
 $C_1([[\text{Kate}]], C_2([[\text{reads}]], [[\text{a book}]])) =$
 $C_1([[\text{Kate}]], C_2([[\text{reads}]], C_3([[\text{a}]], [[\text{book}]])))$



Semantic Analysis with First-Order Logic

(Richard Montague, 1970s)

- Basic approach: Use syntax to guide composition of first-order logical expressions.
- We need:
 - A representation for lexical entries (meaning associated with each word).
 - Rules to perform the composition.

Lambda Expressions

- We need a way to compose FOL formulas from components.
- Lambda notation extends FOL to include expressions of the following form:

$\lambda x.P(x)$

where x is a variable and P is some FOL formula containing x as a *free variable* (not bound by a quantifier).

(think of this a function with a single parameter x)

Combining Lambda Expressions

- Lambda expressions can be applied to other expressions to make new expressions.

$$\frac{\lambda x.P(x) \text{ (A)}}{P(A)} \quad \text{beta-reduction}$$

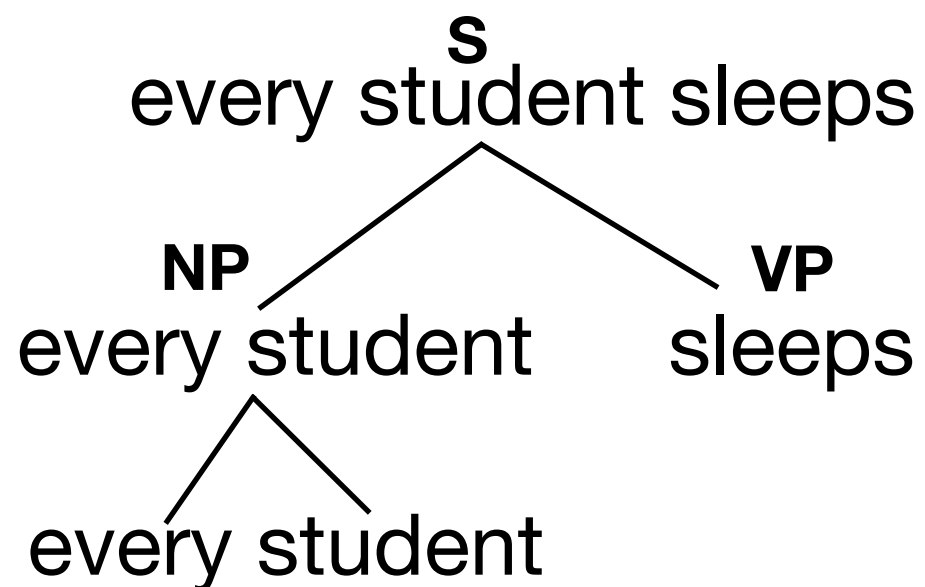
- The body of a lambda expression can be another lambda expression.

$$\frac{\frac{\lambda x.\lambda y.Likes(y,x) \text{ (Alice)}}{\lambda y.Likes(y, Alice)} \text{ (Bob)}}{Likes(Bob, Alice)}$$

- Taking a predicate with multiple arguments and turning it into a sequence of single-argument predicates is called Currying.

Higher-order Functions and Types

- Arguments to lambda expressions can be other lambda expressions (not just constants).



$[[\text{every student sleeps}]] =$
 $[[\text{every student}]] ([[\text{sleeps}]]) =$
 $([[\text{every}]] ([[\text{student}]])) ([[\text{sleeps}]])$

$[[\text{sleeps}]] = \lambda y. \text{sleeps}(y)$ **Type:** $\langle \text{entity}, \text{truth value} \rangle$

$[[\text{student}]] = \lambda z. \text{student}(z)$ **Type:** $\langle e, t \rangle$

$[[\text{every}]] = \lambda P \lambda Q. \forall x (P(x) \rightarrow Q(x))$ **Type:** $\langle \langle e, t \rangle, \langle \langle e, t \rangle, t \rangle \rangle$

Function Application Example

$[[\text{every student}]] = [[\text{every}]] ([[\text{student}]])$

$$\frac{\lambda P \lambda Q. \forall x P(x) \rightarrow Q(x) \quad (\lambda z. \text{student}(z))}{\begin{array}{l} \lambda Q. \forall x (\lambda z. \text{student}(z) (x)) \rightarrow Q(x) \\ \lambda Q. \forall x \text{ student}(x) \rightarrow Q(x) \end{array}}$$

$[[\text{sleeps}]] = \lambda y. \text{sleeps}(y)$

$[[\text{student}]] = \lambda z. \text{student}(z)$

$[[\text{every}]] = \lambda P \lambda Q. \forall x (P(x) \rightarrow Q(x))$

Function Application Example

$[[\text{every student sleeps}]] = [[\text{every student}]] ([[\text{sleeps}]])$

$$\frac{\lambda Q. \forall x \text{ student}(x) \rightarrow Q(x) \quad (\lambda y. \text{sleep}(y))}{\forall x \text{ student}(x) \rightarrow (\lambda y. \text{sleep}(y) (x))}$$
$$\forall x \text{ student}(x) \rightarrow (\lambda y. \text{sleep}(y) (x))$$
$$\forall x \text{ student}(x) \rightarrow \text{sleep}(x)$$

$[[\text{sleeps}]] = \lambda y. \text{sleeps}(y)$

$[[\text{student}]] = \lambda z. \text{student}(z)$

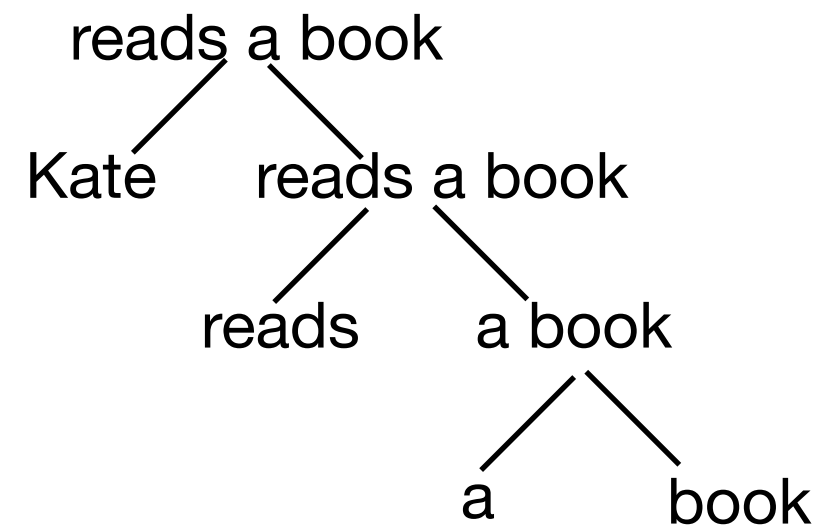
$[[\text{every}]] = \lambda P \lambda Q. \forall x (P(x) \rightarrow Q(x))$

Another Example

$[[\text{Kate}]] = \text{Kate}$

$[[\text{reads}]] = \lambda x. \lambda y. \text{Reads}(y, x)$

$[[\text{a book}]] = \lambda P. \lambda y. \exists x \text{ Book}(x) \wedge P(y, x)$



$[[\text{Kate reads a book}]]$

$= ([[\text{a book}]] ([[\text{reads}]])) ([[\text{Kate}]])$ Apply "a book" to "reads"

$= \exists x \text{ Book}(x) \wedge \text{Reads}(\text{Kate}, x)$

Another Example

$[[\text{reads a book}]] = [[\text{a book}]] ([[\text{reads}]])$

$$\frac{\lambda P. \lambda y. \exists x \text{ Book}(x) \wedge P(y, x) \quad (\lambda s. \lambda t. \text{Reads}(s, t))}{\lambda y. \exists x \text{ Book}(x) \wedge \text{Reads}(y, x)}$$

$[[\text{Kate}]] = \text{Kate}$

$[[\text{reads}]] = \lambda x. \lambda y. \text{Reads}(y, x)$

$[[\text{a book}]] = \lambda P. \lambda y. \exists x \text{ Book}(x) \wedge P(y, x)$

Another Example

$[[\text{Kate reads a book}]] = [[\text{reads a book}]] ([[\text{Kate}]])$

$$\frac{\lambda y. \exists x \text{ Book}(x) \wedge \text{Reads}(y,x) \quad (\text{Kate})}{\exists x \text{ Book}(x) \wedge \text{Reads}(\text{Kate},x)}$$

$[[\text{Kate}]] = \text{Kate}$

$[[\text{reads}]] = \lambda x. \lambda y. \text{Reads}(y,x)$

$[[\text{a book}]] = \lambda P. \lambda y. \exists x \text{ Book}(x) \wedge P(y,x)$

Another Example

$[[\text{every student reads a book}]] = [[\text{every student}]] ([[\text{reads}]])$

$$\frac{\lambda P. \lambda y. \exists x \text{ Book}(x) \wedge P(y, x) \quad (\lambda s. \lambda t. \text{Reads}(s, t))}{\lambda y. \exists x \text{ Book}(x) \wedge \text{Reads}(y, x)}$$

$[[\text{Kate}]] = \text{Kate}$

$[[\text{reads}]] = \lambda x. \lambda y. \text{Reads}(y, x)$

$[[\text{a book}]] = \lambda P. \lambda y. \exists x \text{ Book}(x) \wedge P(y, x)$

Categorical Grammar

- An alternative approach to building phrase structure.
 - Phrases are associated with syntactic *categories* rather than non-terminal symbols.
 - Each lexicon entry is associated with a lexical category.
 - There is a small set of rules to guide combination of these categories in context.
- Inspired by lambda-calculus: Categories are higher-order functions applied to other categories.

Categorical Grammar

- A category is either:
 - An atomic constituent symbol NP, S, ...
 - A single-argument function mapping a desired category to a category.
 - (X / Y) - take category Y as an argument (to the right), return X
 - $(X \setminus Y)$ - take category Y as an argument (to the left), return Y.

Example Lexicon:

```
Mary      := NP
musicals  := NP
likes     := ((S \ NP) / NP)
gives     := (((S \ NP) / NP) / NP)
```

Categorical Grammar: Rules and Derivations

- Forward function application: $>$

$$\frac{(X / Y) \quad Y}{X} >$$

- Backward function application: $<$

$$\frac{Y \quad (X \setminus Y)}{X} <$$

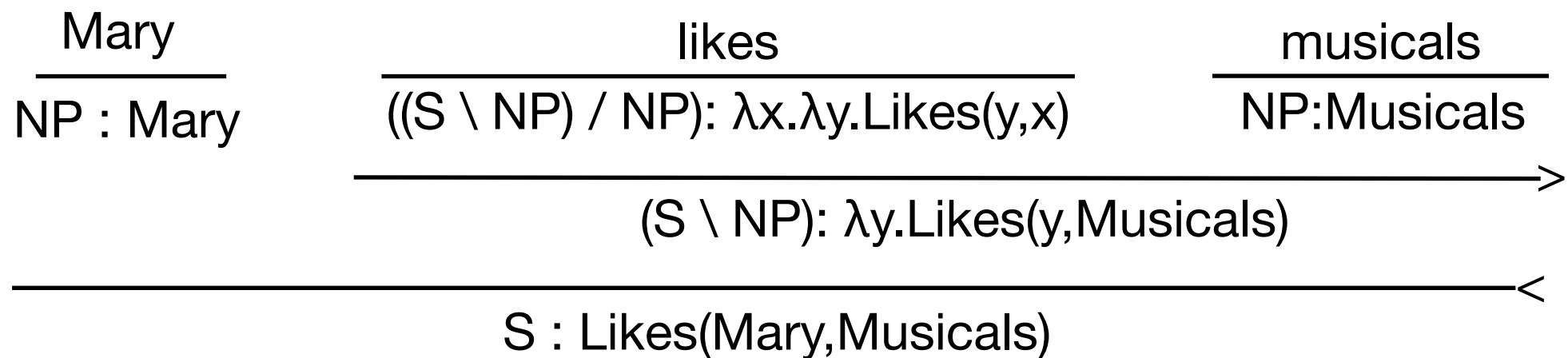
Mary $:=$ NP
 musicals $:=$ NP
 likes $:= ((S \setminus NP) / NP)$

$$\frac{\frac{\text{Mary}}{\text{NP}} \quad \frac{\text{likes}}{((S \setminus NP) / NP)} \quad \frac{\text{musicals}}{\text{NP}}}{(S \setminus NP)} >$$

$$\frac{}{S} <$$

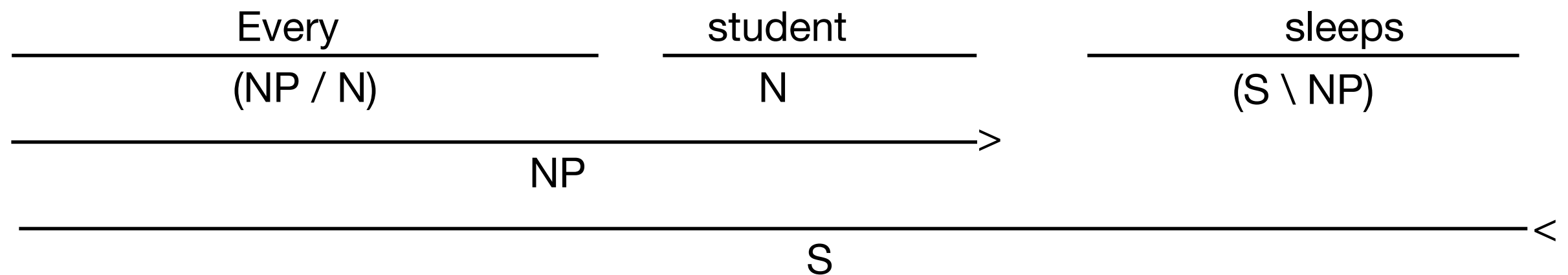
Observation: This is exactly like CFG so far!

Categorical Grammar and Semantic Construction



Mary := NP : Mary
 musicals := NP : Musicals
 likes := ((S \ NP) / NP) : $\lambda x. \lambda y. \text{Likes}(y, x)$

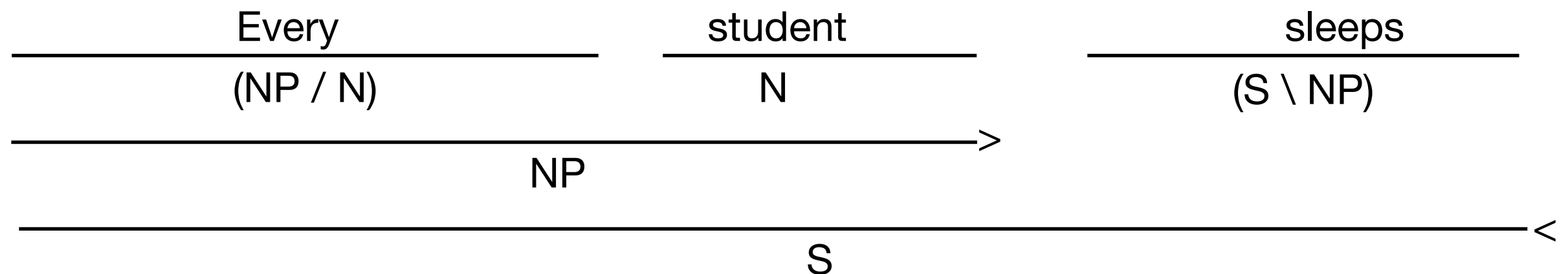
Semantic Construction and Categorial Grammars



Every $:= (NP / N)$
student $:= N$
sleeps $:= (S \setminus NP)$

This corresponds to the following function applications:
sleeps (every (student))

Semantic Construction and Categorial Grammars



Every $:= (NP / N) : \lambda P \lambda Q. \forall x (P(x) \rightarrow Q(x))$ **! Problem, types don't match**
student $:= N : \lambda z. \text{student}(z)$
sleeps $:= (S \setminus NP) : \lambda y. \text{sleeps}(y)$

This corresponds to the following function applications:
sleeps (every (student))

Problem: to compute the logical form we need:
(every (student)) (sleeps)

Semantic Construction and Categorical Grammars

$$\begin{array}{c}
 \frac{\text{Every}}{((S / (S \setminus NP)) / N) : \lambda P \lambda Q. \forall x (P(x) \rightarrow Q(x))} \quad \frac{\text{student}}{N : \lambda z. \text{student}(z)} \quad \frac{\text{sleeps}}{(S \setminus NP) : \lambda y. \text{sleeps}(y)} \\
 \hline
 (S / (S \setminus NP) : \lambda Q. \forall x (\text{student}(x) \rightarrow Q(x))) \rightarrow \\
 \hline
 S : \forall x (\text{student}(x) \rightarrow \text{sleep}(x)) \rightarrow
 \end{array}$$

Every	:=	$((S / (S \setminus NP)) / N)$	:	$\lambda P \lambda Q. \forall x (P(x) \rightarrow Q(x))$
student	:=	N	:	$\lambda z. \text{student}(z)$
sleeps	:=	$(S \setminus NP)$	:	$\lambda y. \text{sleeps}(y)$

We need to change the syntactic category for "every" (type raising)

Combinatory Categorical Grammar (CCG)

- In addition to forward/backward application, we add a number of function *combinators*.

- Forward composition:

$$\frac{(X / Y) \quad (Y / Z)}{(X / Z)} \rightarrow B$$

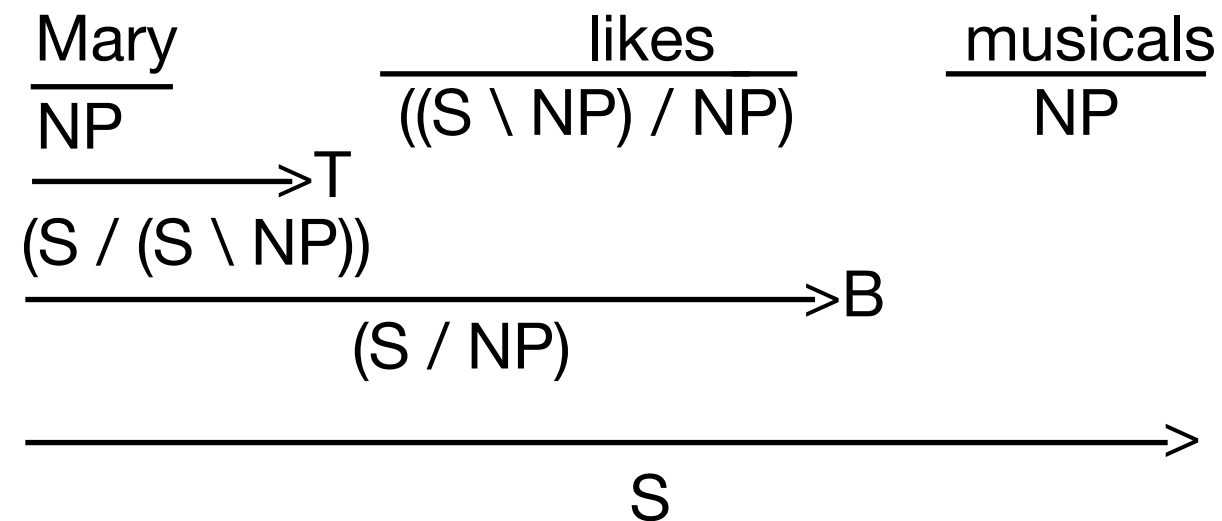
- Backward composition:

$$\frac{(Y \setminus Z) \quad (X \setminus Y)}{(X \setminus Z)} \leftarrow B$$

- Type raising: $\frac{X}{(T / (T \setminus X))} \rightarrow T$ or $\frac{X}{(T \setminus (T / X))} \leftarrow T$

Type Raising Example

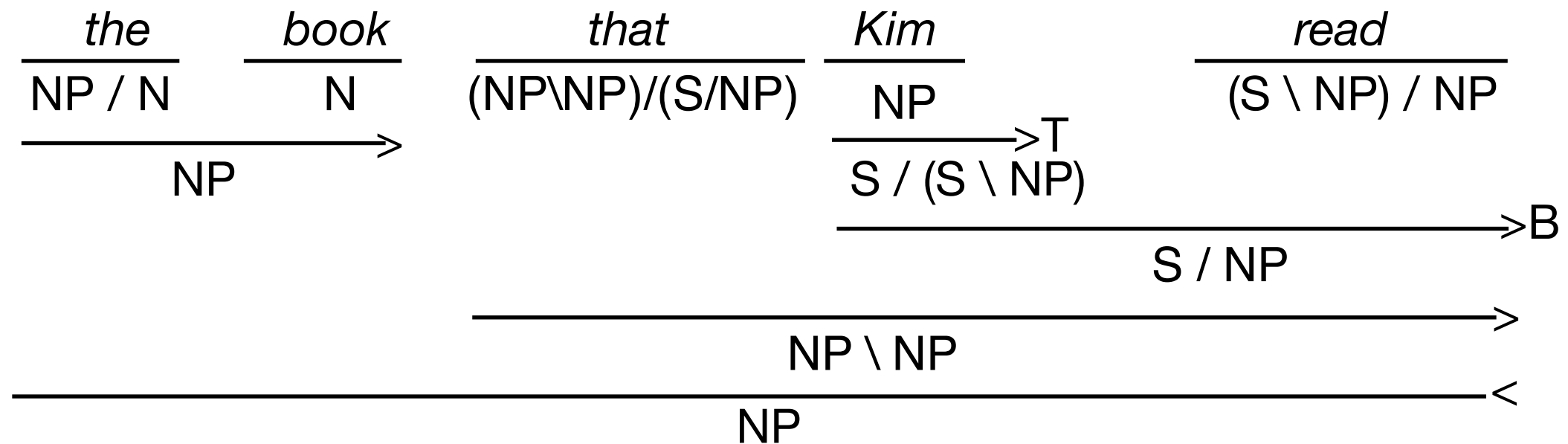
Mary $:=$ NP
musicals $:=$ NP
likes $:= ((S \setminus NP) / NP)$



Note:

- Can process input tokens left-to-right.
- The (S / NP) category does not correspond to a traditional English constituent.

Long-Distance Dependencies in CCG



Note:

- Grammar needs only one lexical entry for *read*!
- Type raising allows us to combine *Kim* with *read* before the object NP is attached. The missing NP is represented in the new category S / NP .

CCG Observations

- CCG has become really popular. CCGBank (Hockenmaier & Steedman 2007) is an automatic conversion of the Penn Treebank to CCG.
- CCG generates the same class of string languages as TAG ("mildly context sensitive").
- Parsing is more expensive (can be done in $O(N^6)$).
 - Efficient greedy Algorithms exist (e.g. Lewis 2015)

Compositional Revisited

- Is natural language really compositional?

'*Sam is **taking** a **shower***'

light verbs

'*He **kicked** the **bucket***'

idioms

'*They **caught up** with them*'

particle verbs

Abstract Meaning Representation (AMR)

(Banarescu et al., 2013)

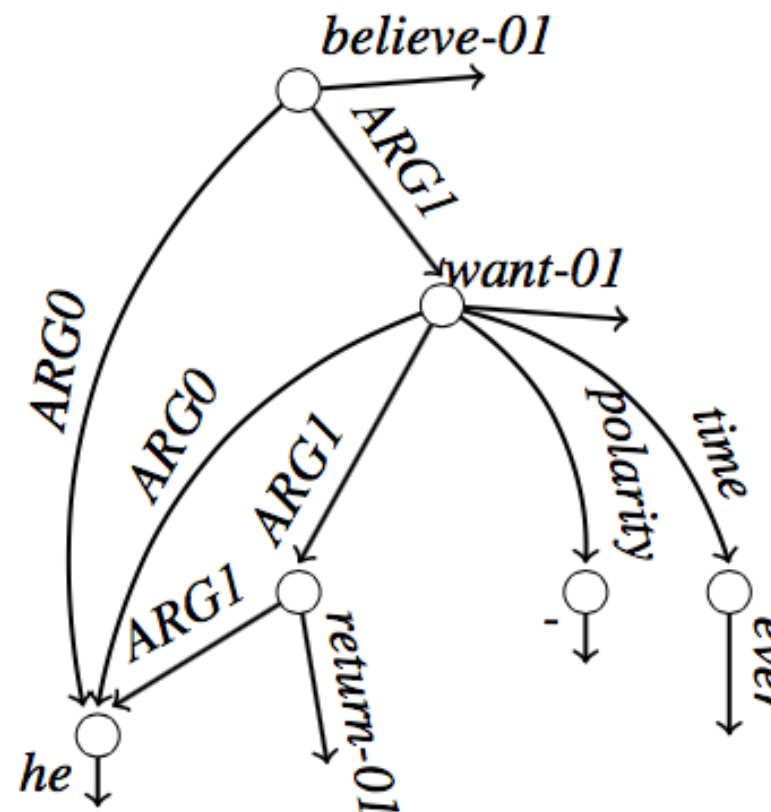
- Uses a single, simple data structure (feature structures / directed graphs) to represent many aspects of meaning.
- Focus on "who does what to whom" but leave out details (tense, quantifiers, etc.)
- This level of abstraction facilitates **consistent, large-scale human annotation**. Goal: build a giant "semantics bank" (comparable to treebanks for syntax).

Abstract Meaning Representation

(Banarescu et al., 2013)

He believed that he would never want to return.

- Basic Idea:



- AMRbank 40k sentences, (includes parts of PropBank and other corpora incl. web discussion forums)
- ISI "*Little Prince Corpus*", 1,562 sentences (amr.isi.edu)

Abstract Meaning Representation (AMR)

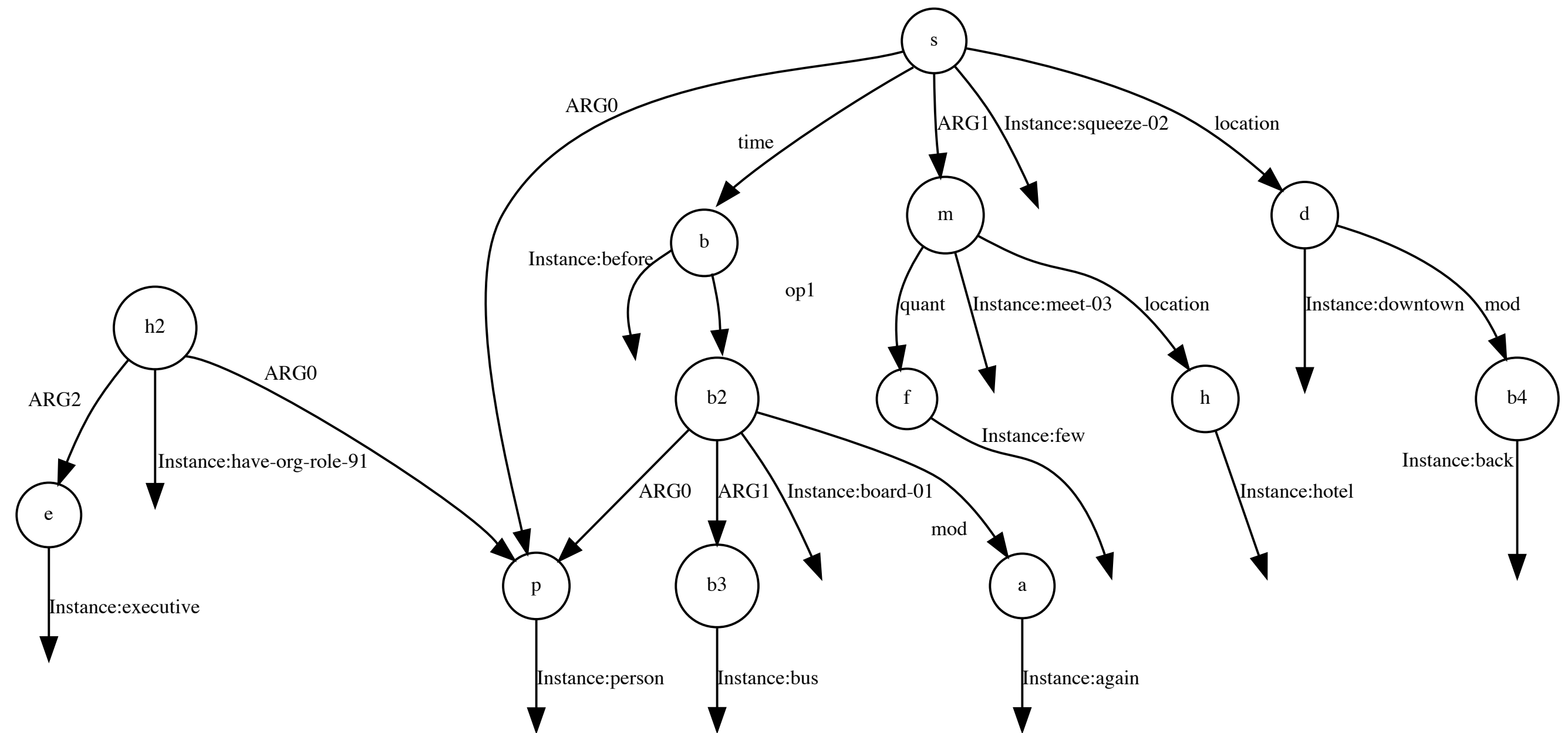
```
(s / squeeze-02
  :ARG0 (p / person
    :ARG0-of (h2 / have-org-role-91
      :ARG2 (e / executive)))

  :ARG1 (m / meet-03
    :location (h / hotel)
    :quant (f / few))

  :location (d / downtown
    :mod (b4 / back))
  :time (b / before
    :op1 (b2 / board-01
      :ARG0 p
      :ARG1 (b3 / bus)
      :mod (a / again))))
```

Back downtown, the execs squeezed in a few meetings at the hotel before boarding the buses again.

Abstract Meaning Representation (AMR)



Back downtown, the execs squeezed in a few meetings at the hotel before boarding the buses again.

AMR Parsing

- English -> AMR, trained automatically on string/graph pairs (AMR data)
- Different approaches:
 - Concept identification + graph-based dependency parsing (JAMR, Flanigan et al. 2014)
 - Transduce dependency tree into AMR (Wang et al. 2015)
 - CCG into logical form + coreference resolution (Artzi 2015)
 - Syntax-based Machine Translation (Pust et al. 2015)
 - Hyperedge Replacement Grammars (Peng 2015, Bauer 2017)
 - Neural encoder-decoder (String-to-graph transduction) (Zhang et al. 2017)

JAMR

(Flanigan et al. 2014)

- Automatically align string spans and graph concepts to obtain a **concept dictionary**.
- For an unseen input sentence:
 - Identify the concepts in the sentence.
 - Identify the relations (edges) between the concepts using a graph-based approach ("spanning graph").
 - Similar to graph-based dependency parsing.

JAMR - Alignments

(Flanigan et al. 2014)

- Uses a set of hand-crafted rules (patterns for named entities, dates, ...)
- Goal: Compute a concept dictionary.

IAEA **accepted** **North Korea** 's **proposal** in **November**.

```
(a / accept-01
  :ARG0 (o / organization
    :ARG0 (n / name
      :op1 "IAEA"))
  :ARG1 (t2 / thing
    :ARG1-of (p / propose-01
      :ARG0 (c / country
        :name (n2 / name
          :op1 "North"
          :op2 "Korea"))))
  :time (d / date-entity
    :month 11))
```

(Schneider & Flanigan, AMR tutorial @ NAACL 2015)

JAMR - Concept ID

The boy wants to visit New York City

Figure 2: A concept labeling for the sentence “The boy wants to visit New York City.”

JAMR - Concept ID

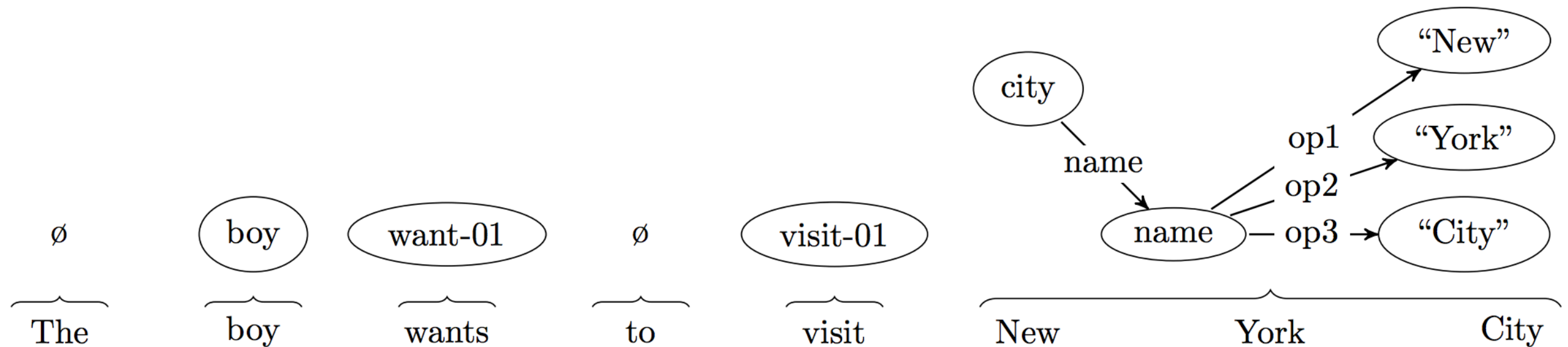
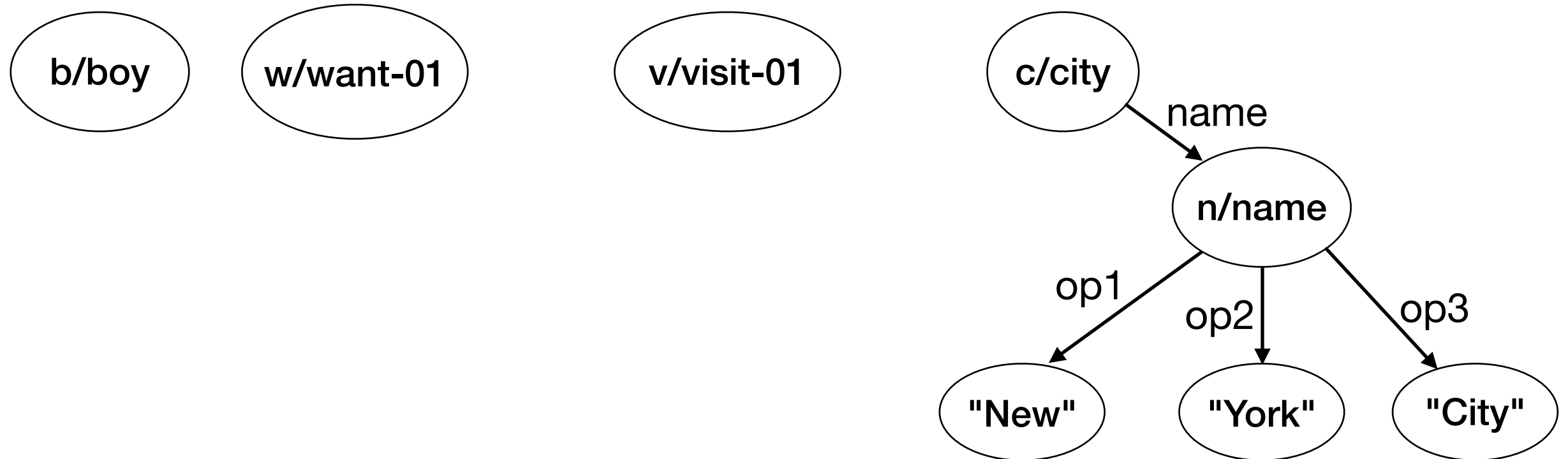


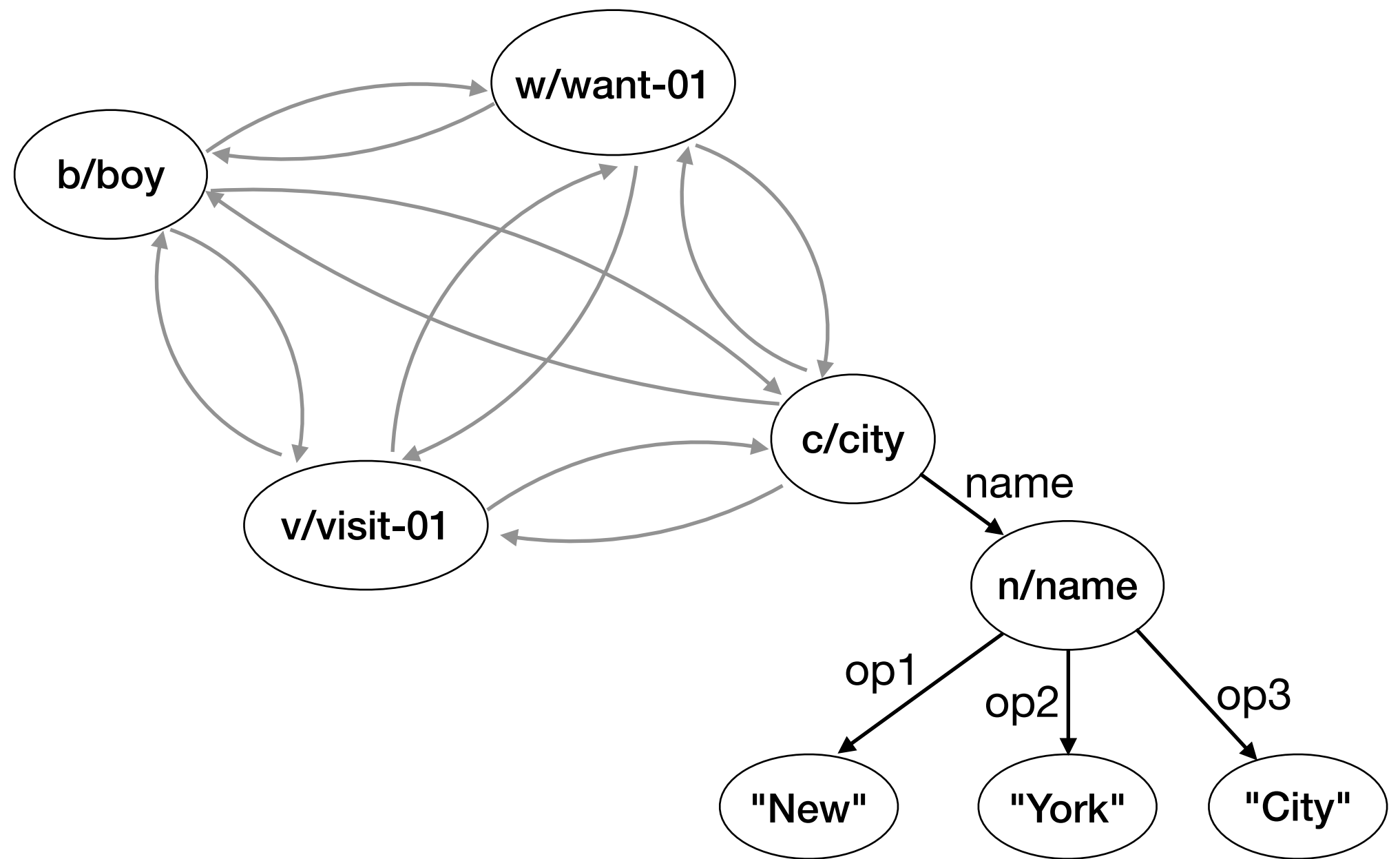
Figure 2: A concept labeling for the sentence "The boy wants to visit New York City."

- Need to compute best span-to-concept assignment.
- But need to consider all different spans.
- Dynamic programming algorithm to solve this.

JAMR - Relation ID

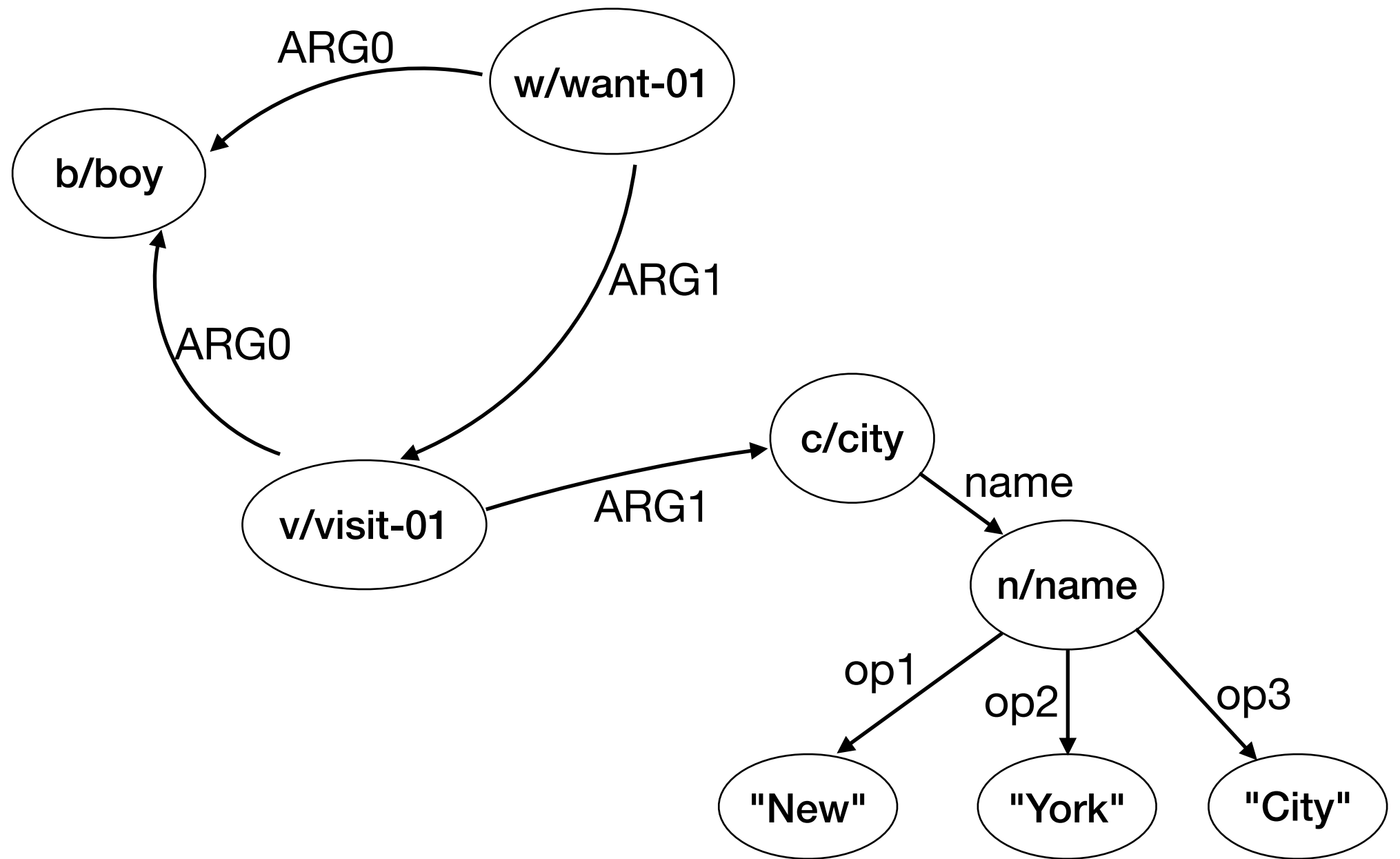


JAMR - Relation ID



Start with completely connected graph (one edge for each relation).

JAMR - Relation ID



Then compute the "Maximum Spanning Connected Subgraph" (MSCC)

AMR Parsing with Synchronous Hyperedge Replacement Grammar

(Bauer 2017)

- String CFGs assemble strings, Hyperedge Replacement Grammars (HRG) assemble graphs.
- Use grammar rules with two matching right-hand sides (string, graph).
 - Parse the string, then follow the derivation and assemble a graph.
- The actual formalism is closer to TAG (lexicalized, allows an adjunction-like operation).
- Rules obtained automatically using alignments.

Recall: String CFG

R1: $S \rightarrow NP VP$

R2: $NP \rightarrow I$

R3: $NP \rightarrow an\ elephant$

R4: $NP \rightarrow my\ pajamas$

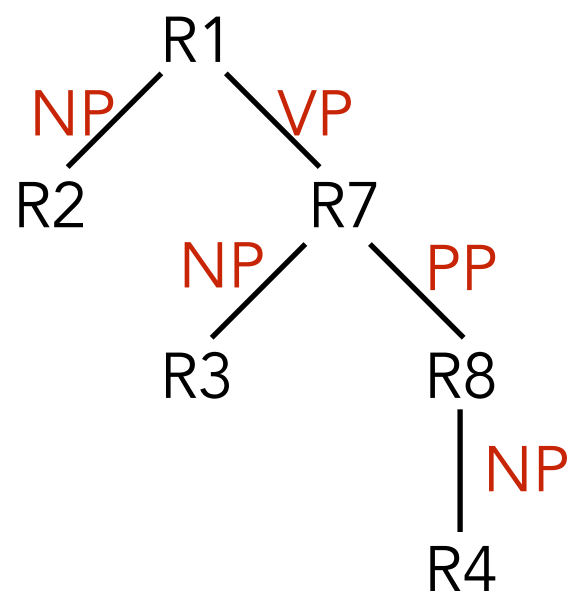
R5: $VP \rightarrow shot\ NP$

R6: $NP \rightarrow NP PP$

R7: $VP \rightarrow shot\ NP PP$

R8: $PP \rightarrow in\ NP$

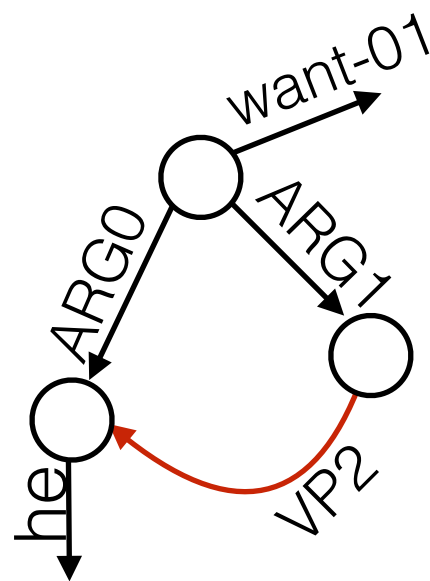
Derivation Tree



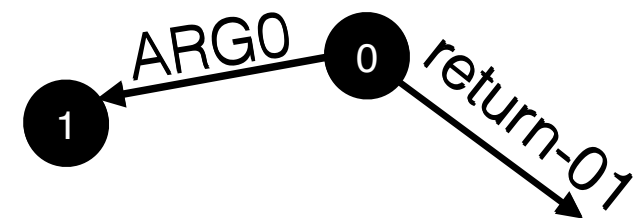
Derived String

I **shot** **an elephant** **in my pajamas**

Hyperedge Replacement Grammar (HRG)

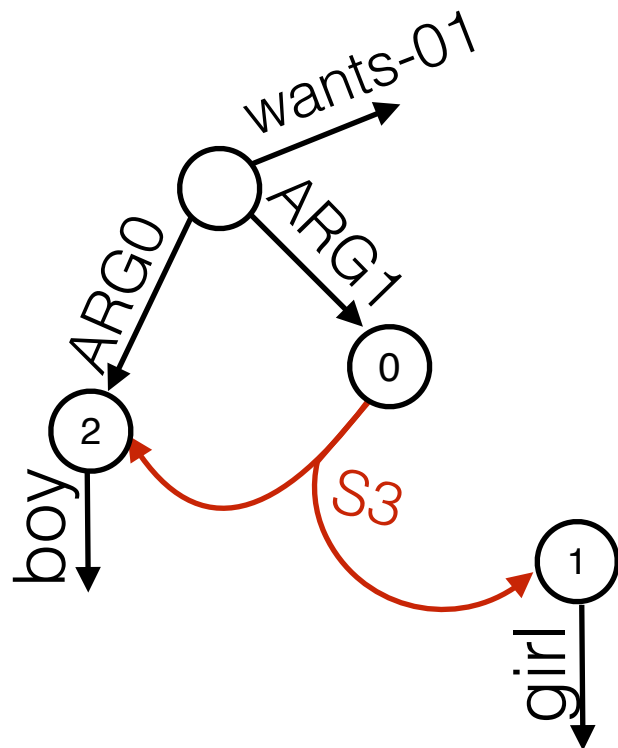


VP2 →



Hyperedge Replacement Grammar (HRG)

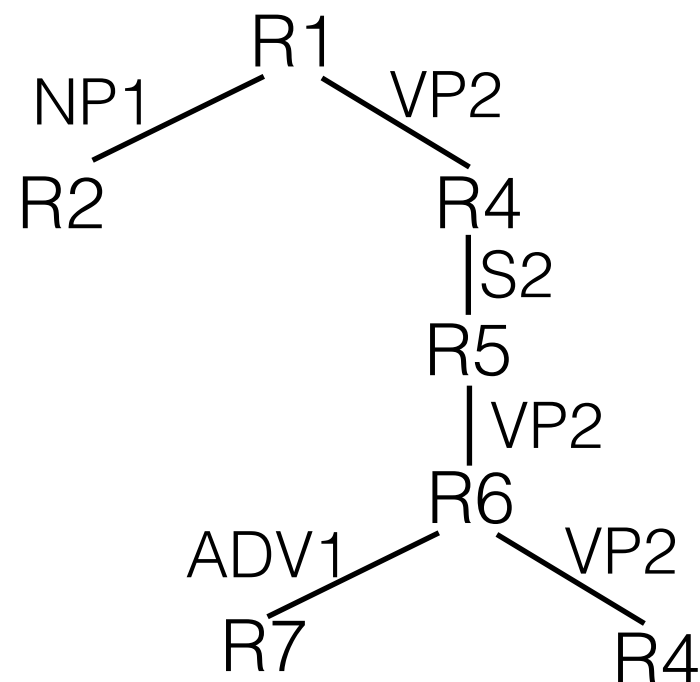
- Hyperedges can have arbitrarily many tentacles. Their endpoints are ordered.



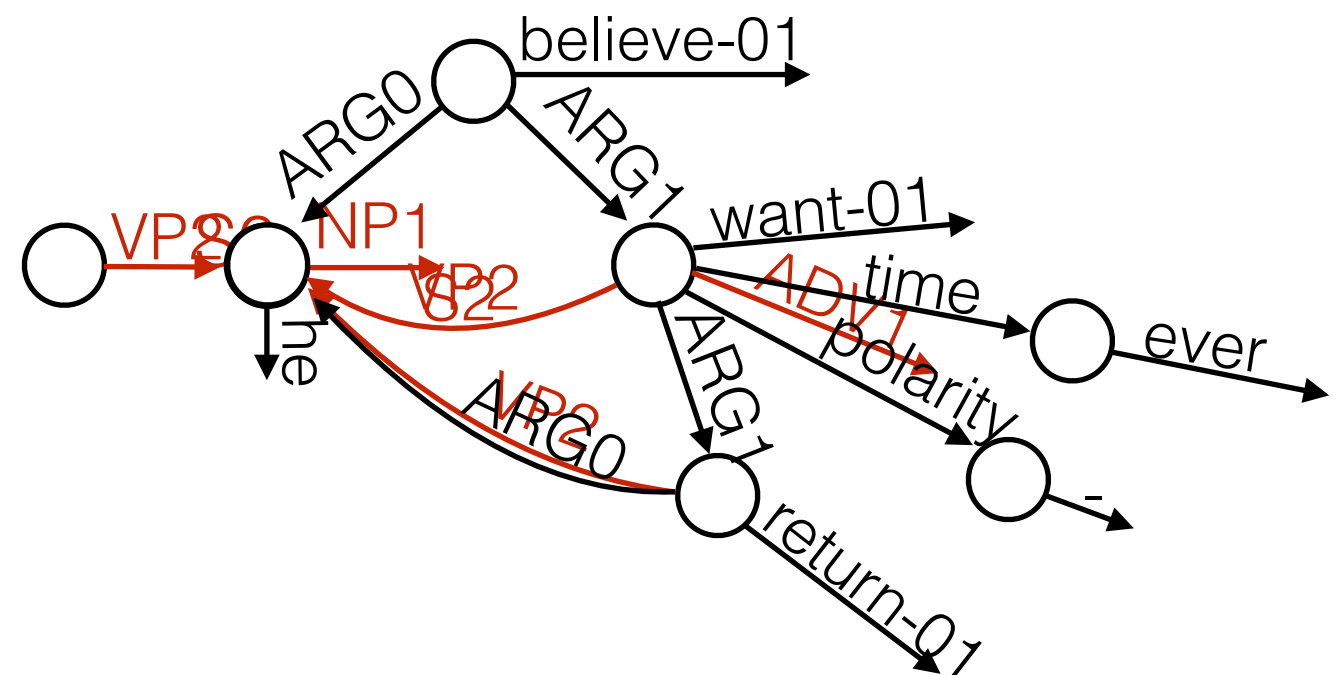
- Number of tentacles: **type** of the hyperedge.
- Terminal / Nonterminal alphabet is also typed.

Hyperedge Replacement Grammar (HRG)

Derivation Tree

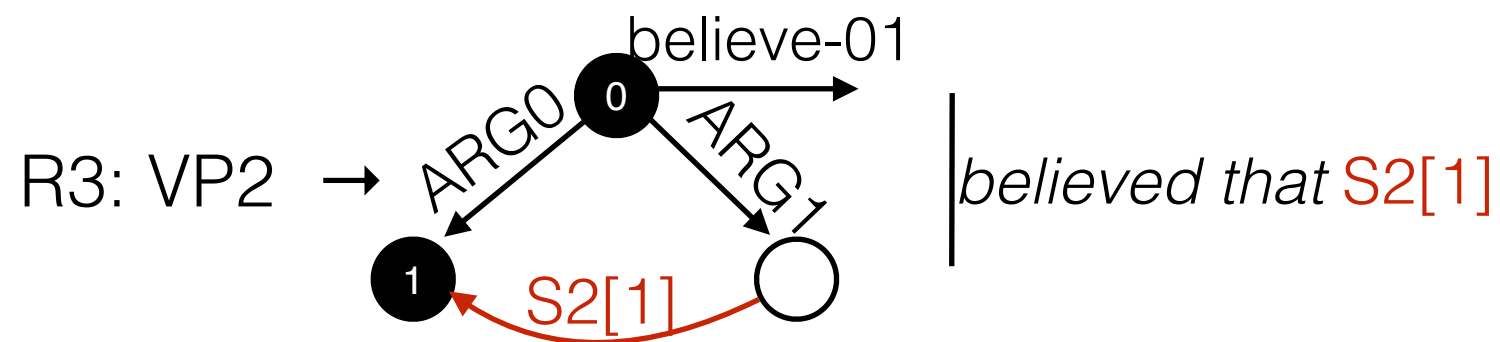


Derived Graph



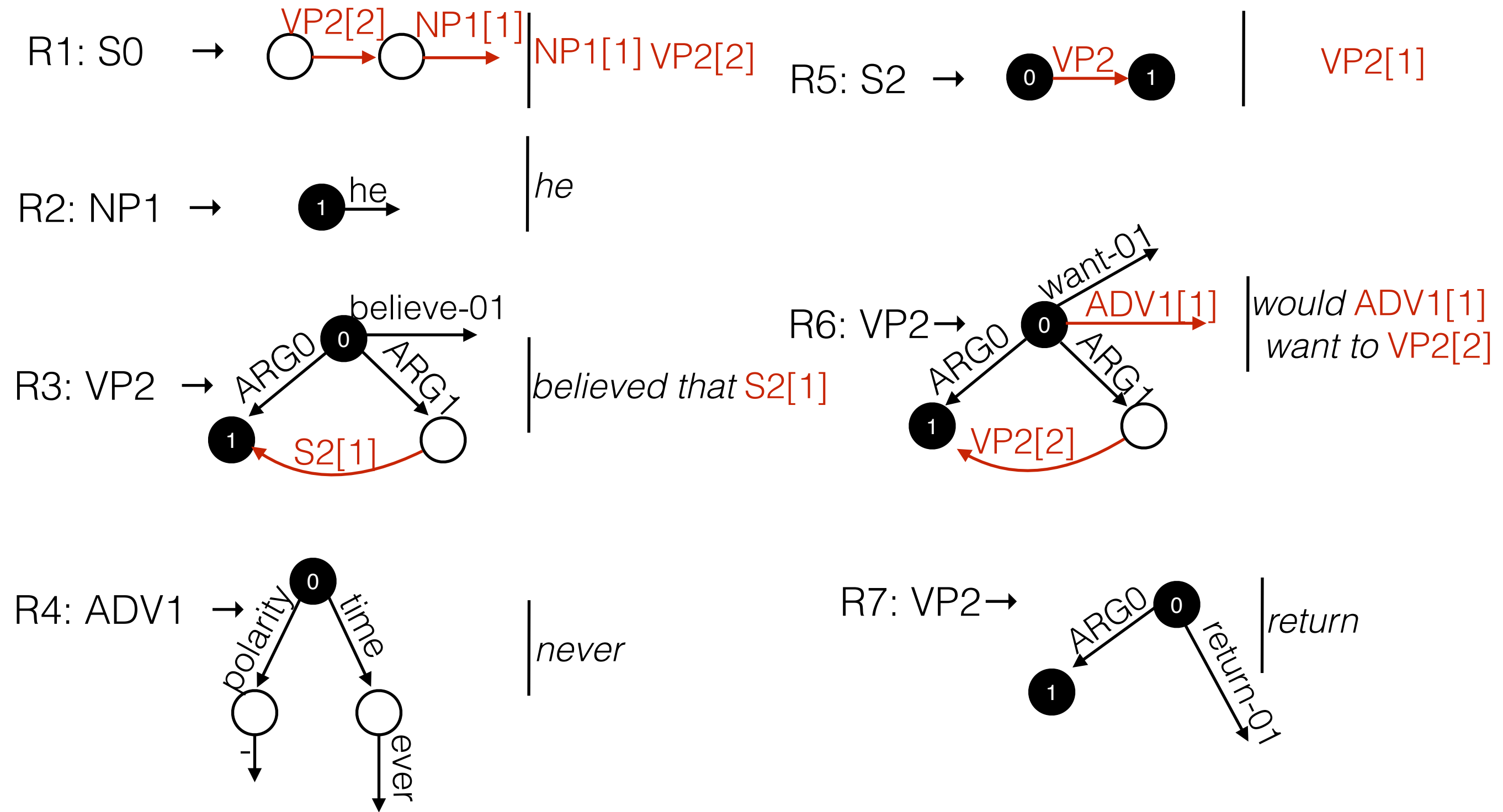
- Polynomial time graph parsing algorithms exist.
(Chiang, Andreas, Bauer, Hermann, Jones & Knight, 2013)

Synchronous Hyperedge Replacement Grammar (SHRG)



- string/graph sides share the same, typed nonterminal alphabet.
- Explicit synchronization (co-indexing).
- Every string derivation is also a valid graph derivation.
- SHRG derive string/graph **pairs**.

Synchronous Hyperedge Replacement Grammar (SHRG)



Synchronous Hyperedge Replacement Grammar (SHRG)

he believed that he would never want to return

